

1. INTRODUCTION

Traffic management has become a major challenge in rapidly growing cities due to the continuous increase in the number of vehicles. Traditional traffic signal systems generally work on fixed timing schedules, which do not consider real-time traffic conditions. Because of this, some roads remain empty while other roads experience heavy congestion.

An intelligent traffic management system is required to overcome these problems. Artificial Intelligence (AI) provides an efficient solution by analyzing real-time traffic data and making automatic decisions. This project focuses on developing an AI-Based Intelligent Traffic Management System using Computer Vision and Deep Learning techniques.

The proposed system uses traffic cameras to capture road conditions. The captured video is processed using OpenCV, and the YOLOv8 deep learning model is used for vehicle detection. The system identifies different types of vehicles, calculates traffic density, and helps in controlling traffic signals according to the current road condition.

This project aims to reduce traffic congestion, minimize waiting time, save fuel consumption, and improve overall transportation efficiency.

2. PROBLEM STATEMENT

In conventional traffic management systems, traffic signals operate according to predefined time intervals. These systems cannot adjust according to the actual number of vehicles present on a road.

Due to this limitation, several problems occur:

- Increased traffic congestion during peak hours.
- Longer waiting time at traffic signals.
- Higher fuel consumption by stationary vehicles.
- Increase in air pollution.
- Difficulty in managing emergency vehicle movement.

To solve these problems, an AI-based system is required that can monitor traffic conditions in real time and automatically make better decisions.

3. OBJECTIVES

The main objectives of the AI-Based Intelligent Traffic Management System are:

- To develop an automated traffic monitoring system using Artificial Intelligence.
- To detect vehicles from real-time traffic videos.
- To count the number of vehicles present on roads.
- To calculate traffic density automatically.
- To provide a concept of smart traffic signal control.
- To reduce traffic congestion and improve road efficiency.
- To minimize human effort in traffic monitoring.

4. SCOPE OF THE PROJECT

The scope of this project is to create an intelligent traffic monitoring solution that can be used in smart cities and modern transportation systems.

The system can be applied for:

- Real-time vehicle monitoring.
- Smart traffic signal management.
- Road traffic analysis.
- Vehicle density estimation.
- Future integration with IoT-based traffic systems.

The project can be further improved by adding advanced features such as emergency vehicle detection, accident detection, and cloud-based traffic monitoring.

5. OVERVIEW OF TECHNOLOGIES USED

5.1 Artificial Intelligence

Artificial Intelligence enables computer systems to perform tasks that normally require human intelligence. In this project, AI is used to analyze traffic conditions and make intelligent decisions.

5.2 Computer Vision

Computer Vision allows computers to understand and process visual information from images and videos. It is used for detecting vehicles through traffic camera footage.

5.3 Deep Learning

Deep Learning is a branch of Machine Learning that uses neural networks to identify complex patterns. In this project, a deep learning-based YOLOv8 model is used for accurate vehicle detection.

5.4 YOLOv8

YOLOv8 (You Only Look Once version 8) is an advanced object detection algorithm used for real-time detection of objects. It provides fast and accurate detection, making it suitable for traffic monitoring applications.

5.5 OpenCV

OpenCV is an open-source computer vision library used for image processing, video handling, and displaying detection results.

6. PROPOSED SYSTEM

The proposed AI-Based Intelligent Traffic Management System works through the following steps:

1. Traffic cameras capture real-time road videos.
2. Video frames are collected and processed.
3. The YOLOv8 model detects vehicles from each frame.
4. Detected vehicles are counted.
5. Traffic density is calculated based on vehicle count.
6. The system provides smart traffic management decisions.

The proposed system provides a flexible approach compared to traditional fixed-time traffic signals because it considers real-time traffic conditions.

7. SYSTEM ARCHITECTURE

The architecture of the AI-Based Intelligent Traffic Management System consists of multiple modules that work together to collect traffic data, process information, detect vehicles, and generate smart decisions.

The complete system architecture is divided into the following layers:

7.1 Data Acquisition Layer

This layer is responsible for collecting real-time traffic information.

Components used:

- Traffic cameras
- Video input sources
- Road monitoring sensors

The cameras capture continuous traffic videos, which are provided as input to the AI processing system.

7.2 Image Processing layer

This layer processes the captured video data using computer vision techniques.

Functions performed:

- Extracting video frames.
- Improving image quality.
- Preparing data for AI detection.

OpenCV is used for handling video processing operations.

7.3 AI Detection layer

The AI detection layer uses the YOLOv8 deep learning model for object detection.

Main functions:

- Detecting vehicles from video frames.
- Identifying vehicle categories.
- Locating vehicles using bounding boxes.

The system can detect different vehicle types such as:

- Cars
- Buses
- Trucks
- Motorcycles

7.4 Decision Making Layer

This layer analyzes the detected vehicle information and makes intelligent decisions.

Functions:

- Calculating the number of vehicles.
- Estimating traffic density.
- Selecting suitable traffic signal timing.

7.5 Control Layer

The control layer manages traffic signal operations according to AI decisions.

Functions:

- Increasing green signal time during heavy traffic.
- Reducing unnecessary waiting time.
- Improving vehicle movement efficiency.

8. SYSTEM WORKING METHODOLOGY

The working process of the proposed system is divided into several steps.

Step 1: Video Data Collection

Traffic cameras capture real-time road videos from different locations. These videos act as input data for the AI system.

Step 2: Frame Extraction

The captured video is divided into individual frames. Each frame is analyzed separately to detect vehicles.

Step 3: Vehicle Detection

The YOLOv8 model processes each frame and identifies vehicles present in the image.

The model provides:

- Object identification.
- Vehicle location.
- Detection confidence score.

Step 4: Vehicle Counting

After detecting vehicles, the system counts the total number of vehicles present in a particular area.

The vehicle count helps in determining traffic conditions.

Step 5: Traffic Density Calculation

The system calculates traffic density based on the number of detected vehicles.

Traffic conditions are classified as:

Low Traffic

When fewer vehicles are present on the road.

Medium Traffic

When the number of vehicles is moderate.

High Traffic

When a large number of vehicles are detected.

Step 6: Smart Signal Decision

According to traffic density, the system decides the required signal timing.

Example:

Traffic Condition| Signal Timing

Low Traffic| Short Green Signal
Medium Traffic| Normal Green Signal
High Traffic| Longer Green Signal

9. ALGORITHM USED

YOLOv8 Object Detection Algorithm

YOLOv8 is a real-time object detection algorithm based on deep learning. It processes an entire image at once and predicts objects along with their locations.

Working Steps of YOLOv8:

1. Input image or video frame is provided to the model.
2. The neural network extracts important features.
3. Objects are identified from extracted features.
4. Bounding boxes are generated around detected objects.
5. Object classes are displayed with confidence values.

YOLOv8 provides fast processing speed and high detection accuracy, making it suitable for real-time traffic monitoring applications.

---10. SYSTEM IMPLEMENTATION

The implementation of the AI-Based Intelligent Traffic Management System is performed using Python programming language. The system combines Computer Vision techniques and the YOLOv8 deep learning model to detect vehicles from traffic videos.

The implementation process includes:

- Loading the trained YOLOv8 model.
- Reading traffic video input.
- Processing video frames.
- Detecting vehicles using AI.
- Displaying detection results.

11. SOFTWARE REQUIREMENTS

Programming Language

Python

Python is used as the main programming language because it provides various libraries for Artificial Intelligence, Machine Learning, and Computer Vision applications.

Libraries Used

OpenCV

OpenCV is used for:

- Reading video files.
- Processing images.
- Displaying output frames.

Ultralytics YOLO

The Ultralytics library provides the YOLOv8 model for real-time object detection.

NumPy

NumPy is used for numerical operations and data processing.

PyTorch

PyTorch provides the deep learning framework required for running AI models.

12. IMPLEMENTATION CODE

The following Python code demonstrates the vehicle detection process using YOLOv8 and OpenCV.

```
import cv2
from ultralytics import YOLO

model = YOLO("models/yolov8n.pt")

cap = cv2.VideoCapture("input/traffic_video.mp4")

while cap.isOpened():
```



```
    success, frame = cap.read()

    if not success:
        break

    results = model(frame)

    annotated_frame = results[0].plot()

    cv2.imshow("AI Traffic Monitoring", annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()

cv2.destroyAllWindows()

---
```

13. CODE EXPLANATION

13.1 Importing Libraries

```
import cv2
from ultralytics import YOLO
```

The required libraries are imported for video processing and AI-based object detection.

OpenCV handles video operations, while YOLO is used for detecting vehicles.

13.2 Loading YOLO Model

```
model = YOLO("models/yolov8n.pt")
```

The YOLOv8 nano model is loaded into the program.

This model is capable of detecting multiple objects from images and videos.

13.3 Capturing Traffic Video

```
cap = cv2.VideoCapture("input/traffic_video.mp4")
```

This command opens the traffic video file and prepares it for frame-by-frame processing.

13.4 Reading Video Frames

```
success, frame = cap.read()
```

The system captures individual frames from the video.

Each frame is analyzed separately by the AI model.

13.5 Vehicle Detection Using AI

```
results = model(frame)
```

The YOLOv8 model processes the frame and detects available objects.

The system identifies vehicles and their positions.

13.6 Displaying Detection Results

```
annotated_frame = results[0].plot()
```

This function adds detection boxes and labels around detected objects.

13.7 Showing Output Window

```
cv2.imshow("AI Traffic Monitoring", annotated_frame)
```

The processed video is displayed with detected vehicles.

13.8 Closing the Program

```
cap.release()
```

```
cv2.destroyAllWindows()
```

These commands release system resources and close the output window after processing.

14. VEHICLE DETECTION MODULE

The vehicle detection module is one of the main components of the system.

The module performs the following tasks:

- Captures traffic video.
- Detects vehicles using YOLOv8.
- Identifies vehicle categories.
- Generates detection output.
- Provides information for traffic density calculation.

The detected vehicles are represented using bounding boxes and labels, which help in visualizing the AI detection process.

15. VEHICLE COUNTING SYSTEM

The vehicle counting system is used to determine the total number of vehicles present on a road. After detecting vehicles using the YOLOv8 model, the system counts the detected objects and uses this information for traffic density estimation.

The vehicle counting process helps in:

- Understanding current traffic conditions.
- Managing traffic signal timing.
- Improving traffic flow.
- Reducing unnecessary waiting time.

16. VEHICLE COUNTING IMPLEMENTATION

The following code detects and counts vehicles from traffic video input.

```

import cv2
from ultralytics import YOLO

model = YOLO("models/yolov8n.pt")

cap = cv2.VideoCapture("input/traffic_video.mp4")

while cap.isOpened():

    success, frame = cap.read()

    if not success:
        break

    results = model(frame)

    vehicle_count = 0

    for result in results:
        for box in result.boxes:

            class_id = int(box.cls[0])

            if class_id in [2, 3, 5, 7]:
                vehicle_count += 1

    print("Number of Vehicles:", vehicle_count)

    annotated_frame = results[0].plot()

    cv2.imshow("Vehicle Detection", annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()

cv2.destroyAllWindows()

---
```

17. VEHICLE COUNTING CODE EXPLANATION

Vehicle Count Initialization

```
vehicle_count = 0
```

A variable is created to store the total number of detected vehicles.

Detecting Vehicle Classes

```
class_id = int(box.cls[0])
```

The YOLO model assigns a class ID to each detected object.

Different IDs represent different object categories.

Vehicle categories include:

- Car
- Motorcycle
- Bus
- Truck

Counting Detected Vehicles

```
vehicle_count += 1
```

Whenever a vehicle is detected, the count increases by one.

This provides the total number of vehicles present in the frame.

18. TRAFFIC DENSITY CALCULATION

Traffic density represents the number of vehicles present on a road at a specific time.

The system calculates traffic density using the vehicle count obtained from AI detection.

Traffic density classification:

Number of Vehicles | Traffic Condition

0 to 10| Low Traffic
11 to 30| Medium Traffic
More than 30| High Traffic

19. DENSITY CALCULATION LOGIC

The traffic condition is determined according to the number of detected vehicles.

```
if vehicle_count <= 10:  
    density = "Low Traffic"  
  
elif vehicle_count <= 30:  
    density = "Medium Traffic"  
  
else:  
    density = "High Traffic"
```

The system automatically identifies whether the road has low, medium, or high traffic.

20. SMART TRAFFIC SIGNAL CONTROL

The smart traffic signal module controls signal timing according to traffic density.

Traditional traffic signals use fixed timing, but this system provides dynamic signal control.

The signal duration is adjusted based on real-time vehicle conditions.

Signal Timing Strategy

Traffic Condition	Green Signal Duration
Low Traffic	20 Seconds
Medium Traffic	40 Seconds
High Traffic	60 Seconds

Working of Smart Signal Control

1. AI detects the number of vehicles.
2. Traffic density is calculated.
3. The system decides suitable green signal timing.
4. Traffic flow is improved by reducing unnecessary waiting.

21. ADVANTAGES OF SMART SIGNAL CONTROL

- Reduces traffic congestion.
- Improves vehicle movement.
- Saves fuel and time.
- Reduces pollution.
- Provides efficient traffic management.

22. HARDWARE REQUIREMENTS

Camera Module

Traffic cameras are used to capture real-time road conditions and provide video input to the AI system.

Processing Unit

The system requires a processing device for running AI models.

Examples:

- Computer/Laptop
- GPU-based system
- Embedded devices

Traffic Signal Controller

Traffic lights receive instructions from the AI system and change signal timing according to traffic conditions.

Sensors

Optional sensors can be used to collect additional traffic information.

23. SOFTWARE REQUIREMENTS

The software requirements define the tools, platforms, and libraries required for developing and running the AI-Based Intelligent Traffic Management System.

23.1 Operating System

The system can be implemented on:

- Windows Operating System
- Linux Operating System

23.2 Development Environment

The project can be developed using:

- Visual Studio Code
- PyCharm
- Jupyter Notebook

These platforms provide an environment for writing, testing, and debugging Python programs.

23.3 Required Python Libraries

OpenCV

OpenCV is used for:

- Video processing.
- Image handling.
- Displaying output results.

Ultralytics

The Ultralytics library provides the YOLOv8 model for real-time object detection.

NumPy

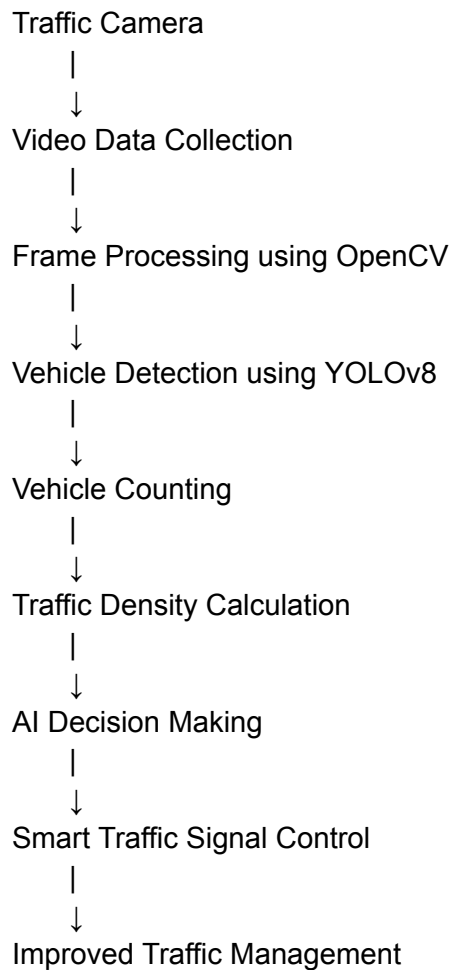
NumPy supports numerical calculations and data processing operations.

PyTorch

PyTorch provides the deep learning framework required for executing AI models.

24. SYSTEM FLOW DIAGRAM

The complete working flow of the proposed system is shown below:



25. SYSTEM TESTING

Testing is performed to verify that the developed system works correctly under different traffic conditions.

The system is tested using different traffic video inputs containing various numbers of vehicles.

26. TESTING PARAMETERS

26.1 Vehicle Detection Accuracy

This parameter checks how accurately the AI model identifies vehicles from traffic videos.

The YOLOv8 model provides efficient object detection with high accuracy.

26.2 Vehicle Counting Accuracy

This parameter verifies whether the system correctly counts the detected vehicles.

Accurate counting helps in proper traffic density estimation.

26.3 Processing Speed

Processing speed determines how quickly the system analyzes video frames.

Fast processing is important for real-time traffic monitoring.

26.4 Signal Response Time

This parameter checks how quickly the traffic signal system responds according to changing traffic conditions.

27. TEST CASES

Test Case	Input Condition	Expected Result
-----------	-----------------	-----------------

Test Case 1	Low traffic video	Low traffic density detected
-------------	-------------------	------------------------------

Test Case 2	Medium traffic video	Medium traffic density detected
-------------	----------------------	---------------------------------

Test Case 3	Heavy traffic video	High traffic density detected
-------------	---------------------	-------------------------------

Test Case 4	Multiple vehicles	Correct vehicle detection and counting
-------------	-------------------	--

28. OUTPUT ANALYSIS

After successful execution of the system, the following outputs are obtained:

- Vehicles are detected automatically from traffic videos.
- Bounding boxes are displayed around detected vehicles.
- Vehicle categories are identified.
- Total vehicle count is generated.
- Traffic density is calculated.
- Smart traffic signal control decisions are generated.

29. PERFORMANCE EVALUATION

The performance of the system is evaluated based on different factors:

Accuracy

The system provides accurate vehicle detection using the YOLOv8 deep learning model.

Efficiency

The automated system reduces manual traffic monitoring efforts and improves decision-making speed.

Reliability

The system continuously analyzes traffic conditions and provides real-time monitoring.

Scalability

The system can be expanded to multiple roads and larger traffic networks.

30. EXPECTED RESULTS

The proposed AI-Based Intelligent Traffic Management System provides:

- Real-time vehicle detection.
- Automatic vehicle counting.

- Traffic density estimation.
- Dynamic signal control.
- Better traffic flow management.

The system demonstrates how Artificial Intelligence can improve traditional traffic management systems.

31. ADVANTAGES OF AI-BASED INTELLIGENT TRAFFIC MANAGEMENT SYSTEM

The proposed AI-Based Intelligent Traffic Management System provides several benefits compared to traditional traffic management systems.

31.1 Reduction in Traffic Congestion

The system analyzes real-time traffic conditions and adjusts signal timing according to vehicle density. This helps in reducing unnecessary traffic jams.

31.2 Time and Fuel Saving

By reducing waiting time at traffic signals, the system helps vehicles save fuel and travel time.

31.3 Reduction in Pollution

Less waiting time and smoother traffic movement reduce harmful vehicle emissions and help in controlling pollution.

31.4 Real-Time Traffic Monitoring

The system continuously monitors traffic conditions using cameras and AI algorithms, providing accurate real-time information.

31.5 Improved Road Safety

AI-based monitoring helps in identifying traffic conditions quickly and supports better road management.

31.6 Reduced Human Effort

The automated decision-making process reduces the requirement for continuous manual traffic monitoring.

32. LIMITATIONS OF THE SYSTEM

Although the system provides an intelligent approach to traffic management, it has some limitations.

32.1 High Initial Cost

Installation of cameras, sensors, and AI processing systems requires investment.

32.2 Weather and Lighting Conditions

Poor lighting, heavy rain, or fog may affect camera-based vehicle detection accuracy.

32.3 Hardware Requirements

Real-time AI processing requires suitable computing power for efficient performance.

32.4 Network Dependency

If cloud-based monitoring is used, a stable network connection is required.

32.5 Detection Challenges

Very crowded roads or blocked vehicle views may reduce detection accuracy.

33. FUTURE SCOPE

The AI-Based Intelligent Traffic Management System can be improved further by adding advanced technologies.

33.1 Emergency Vehicle Detection

The system can be enhanced to detect ambulances, fire trucks, and other emergency vehicles. Traffic signals can provide priority routes for faster movement.

33.2 Accident Detection System

Future versions can include accident detection features to provide quick alerts to emergency services.

33.3 Smart City Integration

The system can be connected with smart city infrastructure to manage traffic across multiple locations.

33.4 Cloud-Based Traffic Monitoring

Traffic data can be stored on cloud platforms for analysis and remote monitoring.

33.5 IoT Integration

IoT sensors can provide additional traffic information and improve system accuracy.

34. POSSIBLE IMPROVEMENTS

Future improvements that can make the system more efficient include:

- Using advanced AI models for higher detection accuracy.

- Adding vehicle tracking functionality.
- Implementing license plate recognition.
- Developing a mobile application for traffic authorities.
- Using real-time traffic prediction models.
- Integrating GPS and navigation systems.

35. FINAL CONCLUSION

The AI-Based Intelligent Traffic Management System provides an effective solution for modern traffic management problems. The system uses Artificial Intelligence, Computer Vision, and Deep Learning techniques to analyze traffic conditions and make intelligent decisions.

The YOLOv8 model enables accurate vehicle detection from real-time traffic videos, while OpenCV helps in processing and displaying the results. The system can detect vehicles, calculate traffic density, and support dynamic traffic signal control.

This project demonstrates how Artificial Intelligence can improve transportation systems by reducing congestion, saving time and fuel, and improving road efficiency.

With further development and integration of IoT, cloud computing, and advanced AI techniques, this system can become an important part of future smart city transportation networks.

36. PROJECT SUMMARY

The complete workflow of the system is:

1. Traffic cameras capture road videos.
2. Video frames are processed using OpenCV.
3. YOLOv8 detects vehicles from frames.
4. Vehicles are counted automatically.
5. Traffic density is calculated.
6. AI makes traffic management decisions.
7. Smart signal control improves traffic flow.

37. REFERENCES

1. Ultralytics YOLO Documentation

- YOLOv8 Object Detection Framework

2. OpenCV Documentation

- Computer Vision and Image Processing Library

3. Python Documentation

- Python Programming Language Reference

4. Research Papers related to:

- Artificial Intelligence in Traffic Management
- Deep Learning-Based Vehicle Detection
- Smart Transportation Systems

PROJECT TITLE

AI-BASED INTELLIGENT TRAFFIC MANAGEMENT SYSTEM USING YOLOv8 AND
COMPUTER VISION

Project Report Completed Successfully

CHAPTER 10: IMPLEMENTATION AND SOURCE CODE

10. IMPLEMENTATION AND SOURCE CODE

10.1 Project Implementation Overview

The implementation phase describes the practical development of the AI-Based Intelligent Traffic Management System. This phase converts the proposed design into a working application using Python programming language, Computer Vision techniques, and the YOLOv8 deep learning model.

The system is implemented in different modules to make the project organized and easy to understand.

The main implementation modules are:

1. Environment Setup Module
2. Video Input Module
3. AI Vehicle Detection Module
4. Vehicle Counting Module
5. Traffic Density Calculation Module
6. Smart Signal Control Module
7. Output Display Module

38.2 Project Environment Setup

The project requires a Python environment with necessary libraries for Artificial Intelligence and Computer Vision operations.

Software Tools Used

- Python Programming Language
- OpenCV Library
- Ultralytics YOLOv8 Framework
- NumPy Library
- PyTorch Deep Learning Framework

38.3 Installation of Required Libraries

The following libraries are required to execute the project:

```
pip install opencv-python
```

```
pip install ultralytics
```

```
pip install numpy
```

```
pip install torch
```

Description

OpenCV is used for video processing and image operations.

Ultralytics provides the YOLOv8 model for object detection.

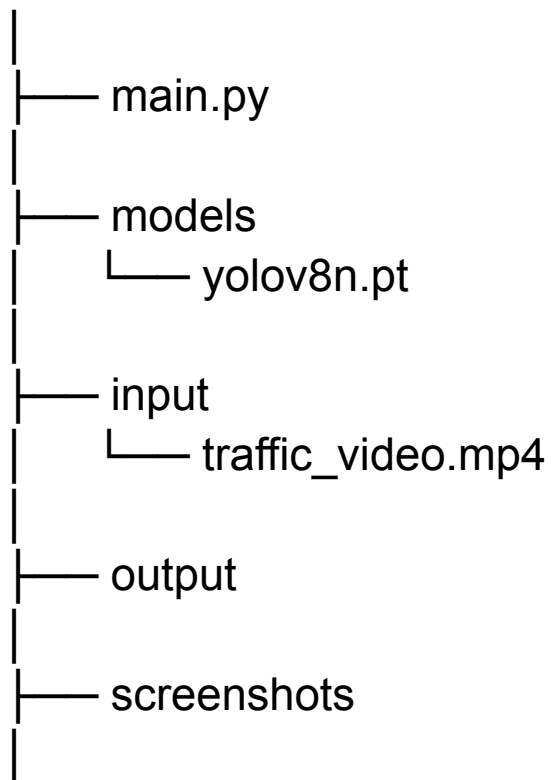
NumPy supports numerical calculations.

PyTorch provides the deep learning framework required for AI model execution.

38.4 Project Folder Structure

The project is organized in the following structure:

AI-Traffic-Management-System



└── requirements.txt

Folder Description

main.py

This file contains the main Python implementation code.

models

This folder stores the YOLOv8 trained model file.

input

This folder contains traffic video files used as system input.

output

This folder stores generated results.

screenshots

This folder contains output images for project documentation.

38.5 Importing Required Libraries

The first step of implementation is importing required libraries.

```
# Import OpenCV library for video processing
import cv2
```

```
# Import YOLO model from Ultralytics framework
from ultralytics import YOLO
```

```
# Import NumPy for numerical operations
import numpy as np
```

Explanation

The imported libraries provide all required functions for:

- Reading traffic videos.
- Processing image frames.
- Running AI-based object detection.
- Performing numerical calculations.

```
# Import OpenCV library for video processing
import cv2
```

```
# Import YOLO model from Ultralytics library
from ultralytics import YOLO
```

```
# Load the pre-trained YOLOv8 model
# This model is used for detecting vehicles from video frames
model = YOLO("models/yolov8n.pt")
```

```
# Open traffic video file
# The video will be processed frame by frame
video = cv2.VideoCapture("input/traffic_video.mp4")
```

```
# Check whether video is successfully opened
if not video.isOpened():
```

```
    print("Unable to open traffic video")
```

```
    exit()
```

```
# Start processing video frames continuously
while True:
```

```
    # Read one frame from video
    status, frame = video.read()
```

```
    # Stop execution when video ends
    if not status:
```

```
break
```

```
# Apply YOLO model on current video frame  
# The model detects objects present in the frame  
detection_result = model(frame)
```

```
# Draw detection boxes and labels on frame  
output_frame = detection_result[0].plot()
```

```
# Display processed frame  
cv2.imshow("AI Vehicle Detection System", output_frame)
```

```
# Press q key to stop the program  
key = cv2.waitKey(1)
```

```
if key == ord("q"):
```

```
    break
```

```
# Release video resources  
video.release()
```

```
# Close all display windows
```

```
cv2.destroyAllWindows()
```

Vehicle Counting Module Using YOLOv8

```
# Import OpenCV library for video processing
```

```
import cv2
```

```
# Import YOLO model for object detection
```

```
from ultralytics import YOLO
```

```
# Load YOLOv8 trained model
```

```
model = YOLO("models/yolov8n.pt")
```

```
# Open traffic video input file
```

```
video = cv2.VideoCapture("input/traffic_video.mp4")
```

```
# Vehicle class names available in COCO dataset
```

```
vehicle_classes = {
```

```
    2: "Car",
```

```
    3: "Motorcycle",
```

```
    5: "Bus",
```



```
7: "Truck"
```

```
}
```

```
# Check video availability  
if not video.isOpened():
```

```
    print("Video file not found")
```

```
    exit()
```

```
# Process video frames continuously  
while True:
```

```
    # Read current frame  
    success, frame = video.read()
```

```
    # Stop when video ends  
    if not success:
```

```
break
```

```
# Detect objects using YOLO model  
results = model(frame)
```

```
# Store detected vehicle count  
vehicle_count = 0
```

```
# Store vehicle category information  
detected_vehicles = []
```

```
# Analyze detection results  
for result in results:
```

```
    # Access detected bounding boxes  
    for box in result.bboxes:
```

```
        # Get object class ID
```

```
class_id = int(box.cls[0])
```

```
# Check whether detected object is a vehicle  
if class_id in vehicle_classes:
```

```
    # Increase vehicle count  
    vehicle_count = vehicle_count + 1
```

```
    # Store detected vehicle name  
    detected_vehicles.append(  
        vehicle_classes[class_id]  
    )
```

```
# Display total number of vehicles  
print("Total Vehicles:", vehicle_count)
```

```
# Display detected vehicle types  
print("Vehicle Types:", detected_vehicles)
```

```
# Draw detection boxes on frame
output_frame = results[0].plot()
```

```
# Show output window
cv2.imshow(
    "Vehicle Counting System",
    output_frame
)
```

```
# Press q to exit program
key = cv2.waitKey(1)
```

```
if key == ord("q"):
```

```
    break
```

```
# Release video resources
video.release()
```

```
# Close all OpenCV windows  
cv2.destroyAllWindows()
```

TRAFFIC DENSITY CALCULATION MODULE

```
# Function to calculate traffic density  
def calculate_density(vehicle_count):
```

```
    # Low traffic condition  
    if vehicle_count <= 10:
```

```
        return "Low Traffic"
```

```
    # Medium traffic condition  
    elif vehicle_count <= 30:
```

```
        return "Medium Traffic"
```

```
    # High traffic condition  
    else:
```

```
        return "High Traffic"
```

```
# Example vehicle count input
```

```
total_vehicle = 25
```

```
# Calculate current traffic density
```

```
density_status = calculate_density(total_vehicle)
```

```
# Display traffic condition
```

```
print("Current Traffic Density:", density_status)
```

```
# Import OpenCV library for video processing
```

```
import cv2
```

```
# Import YOLO model from Ultralytics
```

```
from ultralytics import YOLO
```

```
# Load YOLOv8 model
```

```
# The model detects objects from traffic video
```

```
model = YOLO("models/yolov8n.pt")
```

```
# Open traffic video file
video = cv2.VideoCapture("input/traffic_video.mp4")
```

```
# Vehicle classes from YOLO COCO dataset
```

```
vehicle_classes = {
```

```
    2: "Car",
```

```
    3: "Motorcycle",
```

```
    5: "Bus",
```

```
    7: "Truck"
```

```
}
```

```
# Function to calculate traffic density
```

```
def calculate_density(vehicle_count):
```

```
    # Low traffic condition
```

```
    if vehicle_count <= 10:
```

```
        return "Low Traffic"
```

```
# Medium traffic condition  
elif vehicle_count <= 30:
```

```
    return "Medium Traffic"
```

```
# Heavy traffic condition  
else:
```

```
    return "High Traffic"
```

```
# Function to control signal timing  
def signal_control(density):
```

```
    # Green signal duration for low traffic  
    if density == "Low Traffic":
```

```
        return 20
```



```
# Green signal duration for medium traffic  
elif density == "Medium Traffic":
```

```
    return 40
```

```
# Green signal duration for high traffic  
else:
```

```
    return 60
```

```
# Check video availability  
if not video.isOpened():
```

```
    print("Unable to open video")
```

```
    exit()
```

```
# Start processing video frames
```

```
while True:
```

```
    # Read video frame
```

```
    success, frame = video.read()
```

```
    # Stop when video ends
```

```
    if not success:
```

```
        break
```

```
    # Detect objects using YOLO
```

```
    results = model(frame)
```

```
    # Store vehicle count
```

```
    vehicle_count = 0
```

```
# Store detected vehicle names  
detected_vehicle_list = []
```

```
# Process detection results  
for result in results:
```

```
    # Access every detected object  
    for box in result.bboxes:
```

```
        # Get object class ID  
        class_id = int(box.cls[0])
```

```
        # Check vehicle object  
        if class_id in vehicle_classes:
```

```
            # Increase vehicle count  
            vehicle_count = vehicle_count + 1
```

```
# Store vehicle name
detected_vehicle_list.append(
    vehicle_classes[class_id]
)
```

```
# Calculate traffic density
traffic_status = calculate_density(vehicle_count)
```

```
# Decide signal timing
green_signal = signal_control(traffic_status)
```

```
# Add information on output frame
cv2.putText(
    frame,
    "Vehicles: " + str(vehicle_count),
    (20, 40),
    cv2.FONT_HERSHEY_SIMPLEX,
```

```
1,  
(0, 255, 0),  
2  
)
```

```
cv2.putText(  
    frame,  
    "Density: " + traffic_status,  
    (20, 80),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    1,  
    (0, 255, 0),  
    2  
)
```

```
cv2.putText(  
    frame,  
    "Green Time: " + str(green_signal) + " sec",  
    (20, 120),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    1,  
    (0, 255, 0),
```

```
    2  
)
```

```
# Draw detection boxes  
output = results[0].plot()
```

```
# Display final output  
cv2.imshow(  
    "AI Intelligent Traffic Management System",  
    output  
)
```

```
# Press q to stop execution  
key = cv2.waitKey(1)
```

```
if key == ord("q"):
```

```
    break
```

```
# Release video resource  
video.release()
```

```
# Close all windows  
cv2.destroyAllWindows()
```

Project Execution Steps

The execution process explains the steps required to run the AI-Based Intelligent Traffic Management System successfully.

Software Installation

Before running the project, all required software and libraries must be installed.

Required software:

- Python

- OpenCV
- Ultralytics YOLOv8
- PyTorch
- NumPy

Library Installation

The required libraries can be installed using Python package manager.

```
pip install opencv-python
```

```
pip install ultralytics
```

```
pip install numpy
```

```
pip install torch
```

These libraries provide the required environment for computer vision and artificial intelligence operations.

Model Setup

The YOLOv8 trained model is placed inside the model directory.

Model file:

yolov8n.pt

The model is responsible for detecting vehicles from input traffic videos.

Input Video Setup

The traffic video file is stored inside the input folder.

Example:

input/traffic_video.mp4

The system reads this video and processes each frame for vehicle detection.

Running the Project

The main Python file is executed using the command:

```
python main.py
```

After execution, the system starts processing the traffic video and displays detection results.

Output Analysis

The output generated by the system contains vehicle detection information and traffic management decisions.

Vehicle Detection Result

The YOLOv8 model detects vehicles present in the traffic video.

The output contains:

- Vehicle bounding boxes
- Vehicle categories
- Detection results

Vehicle Counting Result

The system counts the number of detected vehicles from each video frame.

Example:

Detected Vehicles: 25

This information is used for calculating traffic density.

Traffic Density Result

According to the detected vehicle count, the system classifies traffic conditions.

Traffic categories:

- Low Traffic
- Medium Traffic
- High Traffic

Example:

Traffic Condition: High Traffic

Smart Signal Result

The system decides green signal duration according to traffic density.

Example:

Green Signal Duration: 60 Seconds

The dynamic signal timing helps in improving traffic flow.

Result Discussion

The developed system successfully performs real-time traffic monitoring using Artificial Intelligence and Computer Vision.

The obtained results show that the system can:

- Detect vehicles automatically.
- Count vehicles accurately.
- Estimate traffic density.
- Generate intelligent signal timing.

The integration of YOLOv8 with OpenCV makes the system efficient for real-time traffic management applications.

Testing and Performance Evaluation

Testing is an important phase of the AI-Based Intelligent Traffic Management System. The system is tested under different traffic conditions to verify vehicle detection, counting accuracy, and signal control performance.

Test Environment

The system testing is performed using traffic video data containing different numbers of vehicles.

The testing environment includes:

- Python programming environment
- YOLOv8 object detection model
- OpenCV video processing library
- Traffic video input dataset

Test Cases

Test Case| Input Condition| Expected Output

Test Case 1| Low number of vehicles| Low traffic condition detected

Test Case 2| Medium number of vehicles| Medium traffic condition detected

Test Case 3| Large number of vehicles| High traffic condition detected

Test Case 4| Different vehicle categories| Vehicles classified correctly

Performance Evaluation

The performance of the system is evaluated using different parameters.

Vehicle Detection Performance

The YOLOv8 model provides fast and accurate vehicle detection from traffic video frames.

The system can identify different vehicle categories such as:

- Cars
- Motorcycles
- Buses
- Trucks

Vehicle Counting Performance

The vehicle counting module automatically calculates the number of detected vehicles.

Accurate vehicle counting helps the system to estimate traffic density and make better signal decisions.

Processing Speed

The system processes video frames continuously and provides real-time detection results.

Fast processing makes the system suitable for intelligent traffic monitoring applications.

Signal Control Performance

The smart signal control module changes signal timing according to traffic conditions.

Heavy traffic receives longer green signal duration, while low traffic receives shorter duration.

Comparison Between Existing System and Proposed System

Existing Traffic System	Proposed AI-Based System
Fixed signal timing	Dynamic signal timing
Manual monitoring required	Automatic monitoring

Limited traffic analysis| Real-time traffic analysis
Less flexible| AI-based decision making
Higher waiting time| Reduced waiting time

System Benefits After Implementation

After implementation, the proposed system provides:

- Better traffic management.
- Reduced congestion.
- Improved vehicle movement.
- Automatic decision making.
- Efficient use of road resources.

The system demonstrates the capability of Artificial Intelligence in solving real-world transportation problems.

System Design Diagrams Description

The system design diagrams represent the internal working structure and data flow of the AI-Based Intelligent Traffic Management System.

These diagrams help in understanding the interaction between different modules of the system.

System Flow Diagram

The complete workflow of the proposed system is described below:

Traffic Camera



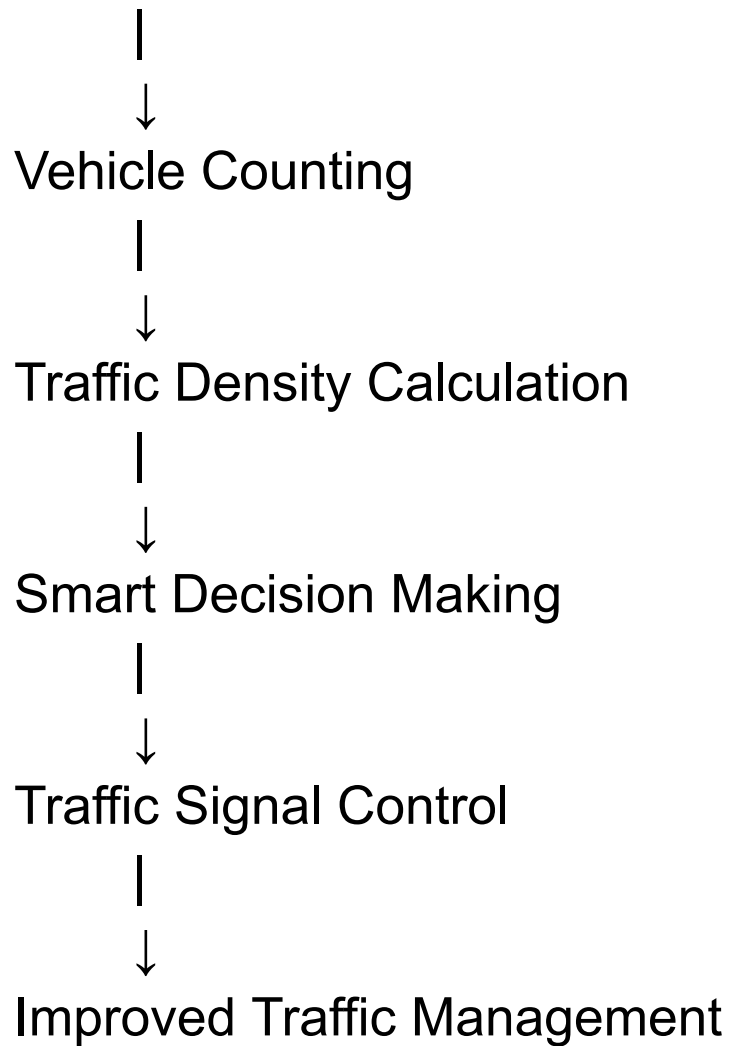
Traffic Video Input



Frame Processing using OpenCV



YOLOv8 Vehicle Detection



Data Flow Description

The data flow of the system starts from traffic cameras that collect real-time road information.

The captured video is transferred to the processing module where frames are extracted and analyzed.

The AI detection module identifies vehicles and sends the information to the decision-making module.

The decision module calculates traffic conditions and generates suitable signal timing.

Module Interaction Description

Video Input Module

This module collects traffic video data from cameras.

Functions:

- Captures road traffic videos.
- Provides input frames to the AI system.
- Transfers data for processing.

Vehicle Detection Module

This module uses YOLOv8 deep learning model for detecting vehicles.

Functions:

- Object identification.
- Vehicle classification.
- Bounding box generation.

Vehicle Counting Module

This module calculates the total number of vehicles detected in each frame.

Functions:

- Counts detected objects.
- Stores vehicle information.

- Provides data for density calculation.

Traffic Density Module

This module determines the traffic condition based on vehicle count.

Functions:

- Analyzes vehicle quantity.
- Classifies traffic level.
- Sends information to signal controller.

Signal Control Module

This module generates traffic signal decisions according to current traffic conditions.

Functions:

- Adjusts green signal duration.
- Reduces unnecessary waiting time.
- Improves traffic movement.

UML Use Case Description

The main actors involved in the system are:

Traffic Monitoring System

Responsibilities:

- Captures traffic information.
- Processes video data.
- Detects vehicles.

AI Processing Unit

Responsibilities:

- Runs YOLOv8 model.
- Performs vehicle analysis.
- Calculates traffic density.

Traffic Signal Controller

Responsibilities:

- Receives AI decisions.
- Changes signal timing.
- Controls traffic flow.

Activity Flow Description

The activity flow of the system is:

1. Start system.
2. Capture traffic video.
3. Process video frames.
4. Detect vehicles using AI.

5. Count vehicles.
6. Calculate traffic density.
7. Generate signal timing.
8. Display output.
9. Stop system.

Appendix

The appendix contains additional information related to the project implementation.

It includes:

- Source code details.
- Installation commands.
- Project structure.
- Additional configuration information.
- Testing details.

The appendix helps users understand and maintain the developed system.

```
# Import required libraries
import cv2
from ultralytics import YOLO
import time

# Load YOLOv8 detection model
model = YOLO("models/yolov8n.pt")

# Open traffic video file
cap = cv2.VideoCapture("input/traffic_video.mp4")

# Vehicle class names
vehicle_names = {

    2: "Car",
    3: "Motorcycle",
    5: "Bus",
    7: "Truck"

}
```

```
# Function to calculate traffic density
```

```
def get_traffic_density(count):
```

```
    if count <= 10:
```

```
        return "Low Traffic"
```

```
    elif count <= 30:
```

```
        return "Medium Traffic"
```

```
    else:
```

```
        return "High Traffic"
```

```
# Function for signal timing decision
```

```
def get_signal_time(condition):
```

```
    if condition == "Low Traffic":
```

```
return 20
```

```
elif condition == "Medium Traffic":
```

```
    return 40
```

```
else:
```

```
    return 60
```

```
# Check video status
```

```
if cap.isOpened() == False:
```

```
    print("Video not available")
```

```
    exit()
```

```
# Start processing video  
while True:
```

```
    # Read frame  
    ret, frame = cap.read()
```

```
    if ret == False:
```

```
        break
```

```
    # Detect objects  
    results = model(frame)
```

```
    # Initialize vehicle counter  
    count = 0
```

```
# Store vehicle information  
detected = []
```

```
# Process detection result  
for result in results:
```

```
    for box in result.bboxes:
```

```
        class_id = int(box.cls[0])
```

```
        if class_id in vehicle_names:
```

```
            count = count + 1
```

```
            detected.append(  
                vehicle_names[class_id]  
            )
```

```
# Calculate traffic condition
density = get_traffic_density(count)
```

```
# Calculate signal duration
signal_time = get_signal_time(density)
```

```
# Display information on screen
cv2.putText(
    frame,
    "Vehicle Count: " + str(count),
    (20, 40),
    cv2.FONT_HERSHEY_SIMPLEX,
    1,
    (0,255,0),
    2
)
```



```
cv2.putText(  
    frame,  
    "Traffic: " + density,  
    (20, 80),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    1,  
    (0,255,0),  
    2  
)
```

```
cv2.putText(  
    frame,  
    "Signal Time: " + str(signal_time),  
    (20,120),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    1,  
    (0,255,0),  
    2  
)
```

```
# Draw detection boxes  
output = results[0].plot()
```

```
# Show output  
cv2.imshow(  
    "AI Traffic Management System",  
    output  
)
```

```
# Exit using q key  
if cv2.waitKey(1) & 0xFF == ord("q"):  
  
    break
```

```
# Release resources  
cap.release()
```

```
cv2.destroyAllWindows()
```

```
# Import required libraries
```

```
import cv2
```

```
from ultralytics import YOLO
```

```
# Load YOLOv8 model
```

```
model = YOLO("models/yolov8n.pt")
```

```
# Open traffic video
```

```
cap = cv2.VideoCapture("input/traffic_video.mp4")
```

```
# Store previous vehicle positions
```

```
vehicle_positions = {}
```

```
# Unique vehicle ID counter
```

```
vehicle_id = 0
```

```
# Vehicle classes
vehicle_classes = {

    2: "Car",
    3: "Motorcycle",
    5: "Bus",
    7: "Truck"

}
```

```
# Function to calculate distance between two points
def calculate_distance(point1, point2):

    distance = (
        (point1[0] - point2[0]) ** 2 +
        (point1[1] - point2[1]) ** 2
    )

    return distance
```

```
# Process video frames  
while True:
```

```
    # Read frame  
    success, frame = cap.read()
```

```
    if success == False:
```

```
        break
```

```
    # Detect objects  
    results = model(frame)
```

```
    # Current frame vehicle data  
    current_objects = {}
```

```
# Extract detection results  
for result in results:
```

```
    for box in result.bboxes:
```

```
        class_id = int(box.cls[0])
```

```
        # Check vehicle category  
        if class_id in vehicle_classes:
```

```
            # Get bounding box coordinates  
            coordinates = box.xyxy[0]
```

```
            x1 = int(coordinates[0])
```

```
            y1 = int(coordinates[1])
```

```
            x2 = int(coordinates[2])
```

```
y2 = int(coordinates[3])
```

```
# Calculate center point  
center_x = int((x1 + x2) / 2)
```

```
center_y = int((y1 + y2) / 2)
```

```
current_objects[vehicle_id] = {  
    "type": vehicle_classes[class_id],  
    "center": (center_x, center_y)  
}
```

```
vehicle_id = vehicle_id + 1
```

```
# Update vehicle positions  
vehicle_positions = current_objects
```

```
# Draw vehicle information
for object_id, data in vehicle_positions.items():
```

```
    position = data["center"]
```

```
    cv2.circle(
        frame,
        position,
        5,
        (0,255,0),
        -1
    )
```

```
    cv2.putText(

        frame,

        data["type"] + " ID:" + str(object_id),
```



```
    position,  
  
    cv2.FONT_HERSHEY_SIMPLEX,  
  
    0.6,  
  
    (255,255,255),  
  
    2  
  
)
```

```
# Display tracking output  
cv2.imshow(  
    "Vehicle Tracking System",  
    frame  
)
```

```
# Exit program
```

```
if cv2.waitKey(1) & 0xFF == ord("q"):
```

```
    break
```

```
# Release video
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

```
# Import required libraries
```

```
import cv2
```

```
from ultralytics import YOLO
```

```
# Load YOLOv8 model
```

```
model = YOLO("models/yolov8n.pt")
```

```
# Open traffic video
```

```
cap = cv2.VideoCapture("input/traffic_video.mp4")
```

```
# Emergency vehicle classes
# These classes can be extended according to
dataset
emergency_keywords = [

    "ambulance",
    "fire truck",
    "police"

]
```

```
# Function to check emergency vehicle
def check_emergency(vehicle_name):

    name = vehicle_name.lower()

    for keyword in emergency_keywords:

        if keyword in name:
```

```
return True
```

```
return False
```

```
# Process video frames  
while True:
```

```
    success, frame = cap.read()
```

```
    if success == False:
```

```
        break
```

```
# Detect objects  
results = model(frame)
```

```
emergency_detected = False
```

```
# Analyze detected objects  
for result in results:
```

```
    for box in result.bboxes:
```

```
        class_id = int(box.cls[0])
```

```
        confidence = float(box.conf[0])
```

```
# Get detected object name  
        object_name = model.names[class_id]
```

```
# Check emergency vehicle
if check_emergency(object_name):

    emergency_detected = True
```

```
# Display emergency priority message
if emergency_detected:
```

```
    cv2.putText(

        frame,

        "Emergency Vehicle Priority Activated",

        (20,50),

        cv2.FONT_HERSHEY_SIMPLEX,

        1,
```

```
(0,0,255),  
  
3  
  
)
```

else:

```
cv2.putText(  
  
    frame,  
  
    "Normal Traffic Control",  
  
    (20,50),  
  
    cv2.FONT_HERSHEY_SIMPLEX,  
  
    1,  
  
    (0,255,0),
```

3

)

```
# Display output  
cv2.imshow(
```

```
    "Emergency Vehicle Priority System",
```

```
    frame
```

```
)
```

```
# Exit condition
```

```
if cv2.waitKey(1) & 0xFF == ord("q"):
```

```
    break
```



```
# Release resources  
cap.release()
```

```
cv2.destroyAllWindows()
```